# Markov Chains, Part 2

Simulation, forecasting, and the long run

# Where We Left Off

- Tuesday: sequences → **state space** → transition counts → **transition matrix**
- We raised the **order** of the Bach chain by making the state a chord pair
- We ended on one question: can every state reach every other state? (**connected chains**)

Today

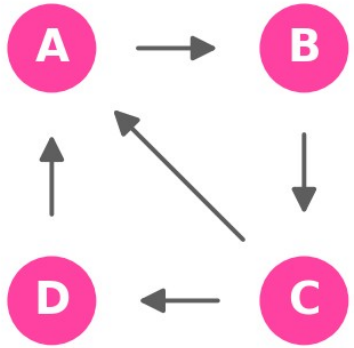
- Recap the vocabulary
- Connected chains: reachability, traps, and cycles
- **Simulation**: draw one random path, a **random walk**
- **Forecasting**: carry the whole probability distribution forward
- The **stationary distribution**: where the chain settles in the long run

Today: recap → connected chains → simulation → forecasts → stationary distribution

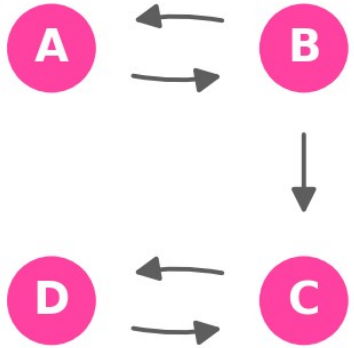
# Reachability and Connectivity

- State j is **reachable** from i if some path i → … → j uses only moves with $P > 0$. A direct move is not required
- A chain is **strongly connected** (**irreducible**) if every state is reachable from every other state
- It is **weakly connected** if the graph is connected once you ignore arrow directions: you can get stuck on one side

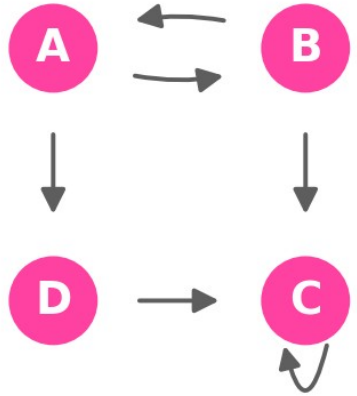
Strongly connected
(irreducible)




Weakly connected:
{A, B} → {C, D}, no way back




C is absorbing:
every walk ends there




Today: recap → connected chains → simulation → forecasts → stationary distribution

# Absorbing States are dead ends

- A state is **absorbing** if $P_{ii} = 1$: once there, the chain never leaves
- A state with **no observed departures** has a row of zeros. That row is not a probability distribution
  - In data this happens at sequence ends: a chord pair that only ever closes a chorale
- The data cannot tell us what happens next there. Pick a policy and **document it**:
  - e.g. make the state absorbing by setting its diagonal entry to 1
  - This is an assumption, and it can change the long-run behavior
- Do not silently drop such states and renormalize the others

```
# one policy: unobserved departures ->
absorbing
P = transition_probabilities(counts)
dead = counts.sum(axis=1) == 0
P[dead, dead] = 1          # stay put forever

assert np.allclose(P.sum(axis=1), 1)
```

Unobserved is not impossible: a zero count is evidence about our sample, not proof about the world

# Is the Bach Chain Connected?

```
import networkx as nx

G = nx.from_numpy_array(bach_counts > 0,
                        create_using=nx.DiGraph)
nx.is_strongly_connected(G)

comps = nx.strongly_connected_components(G)
sizes = [len(c) for c in comps]

# repeat with bach_pair_counts
```

| | Order 1: chords | Order 2: pairs |
|---|---|---|
| **States** | 99 | 757 |
| **Strongly connected?** | Yes | No |
| **Components** | 1 | 15 |
| **Largest component** | 99 | 743 |
| **States never left** | 0 | 3 |

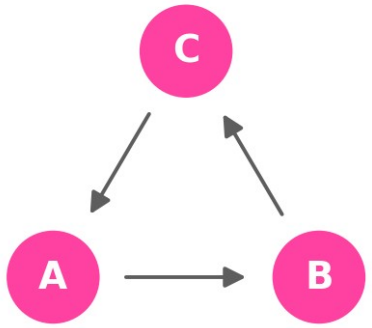
- Order 1: every chord can reach every other chord. 83 of the 99 chords also have a self-loop (a chord repeated)
- Order 2: 3 chord pairs appear only at the end of a chorale, so they have no observed departures, and a few small groups of pairs cannot get back to the rest
- **We use the first-order chain for forecasting**: it is connected and every row is a probability distribution

Today: recap → connected chains → simulation → forecasts → stationary distribution

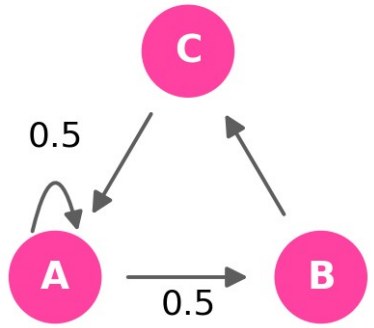
# One More Property: Periodicity

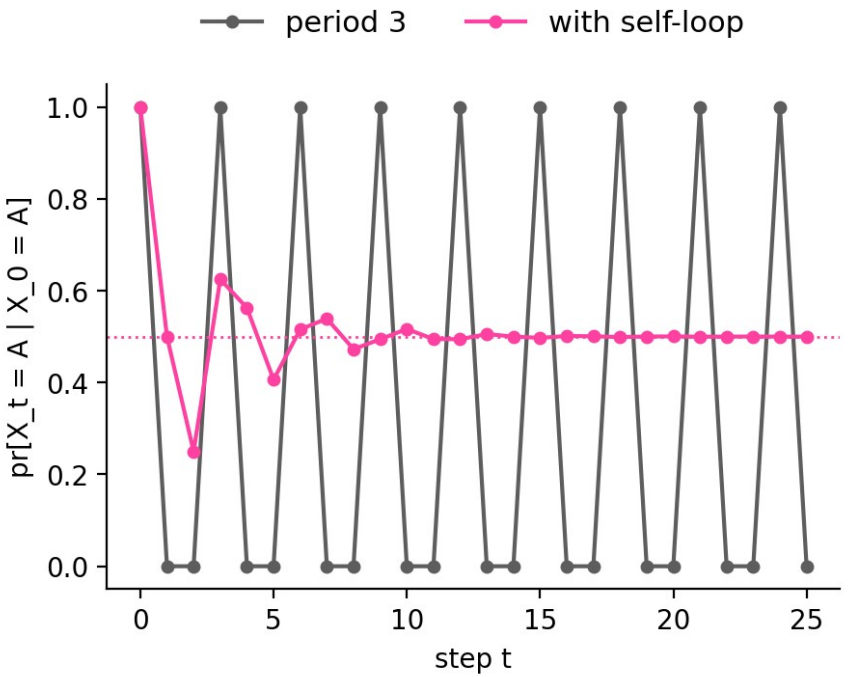
- A connected chain can still **cycle**: in A → B → C → A, the walk returns to A only every 3 steps
- The **period** of a state is the greatest common divisor of the possible return times. A chain is **aperiodic** if every period is 1
- In a connected chain, **one self-loop is enough** for aperiodicity: the walk can wait a step and break the rhythm

Period 3: A only at t = 0, 3, 6, …




One self-loop: aperiodic




Today: recap → connected chains → simulation → forecasts → stationary distribution

# Simulation and Random Walks

# Simulation: Draw One Possible Path

- With a state space and a transition matrix we can **simulate** the chain:
  - 1. Pick an initial state
  - 2. Draw the next state from the current state's row of P
  - 3. Move there and repeat step 2
- The resulting path is a **random walk** on the state graph
- Each run gives a different path. The randomness is in step 2
- Stop if the current row has no departures: there is nothing to draw from

```
def simulate(states, P, initial_state, n_steps, seed=100):
    rng = np.random.default_rng(seed)
    index = states.index(initial_state)
    path = [initial_state]
    for _ in range(n_steps):
        outgoing = P[index]
        if outgoing.sum() == 0:
            print('No observed departures; stopping.')
            break
        index = rng.choice(len(states), p=outgoing)
        path.append(states[index])
    return path
```

# Bach: Independent Draws vs. the Markov Chain

- **Independent**: draw 21 chords from the overall chord frequencies, ignoring order
- **First-order chain**: start at C and draw 20 transitions from the rows of P

Independent chords

`F#m → E → C → A → Gm → E → F# → G → Em → Bb → Gm → C → E → E → E → D7 → Gm → D → F → D → Gm7`

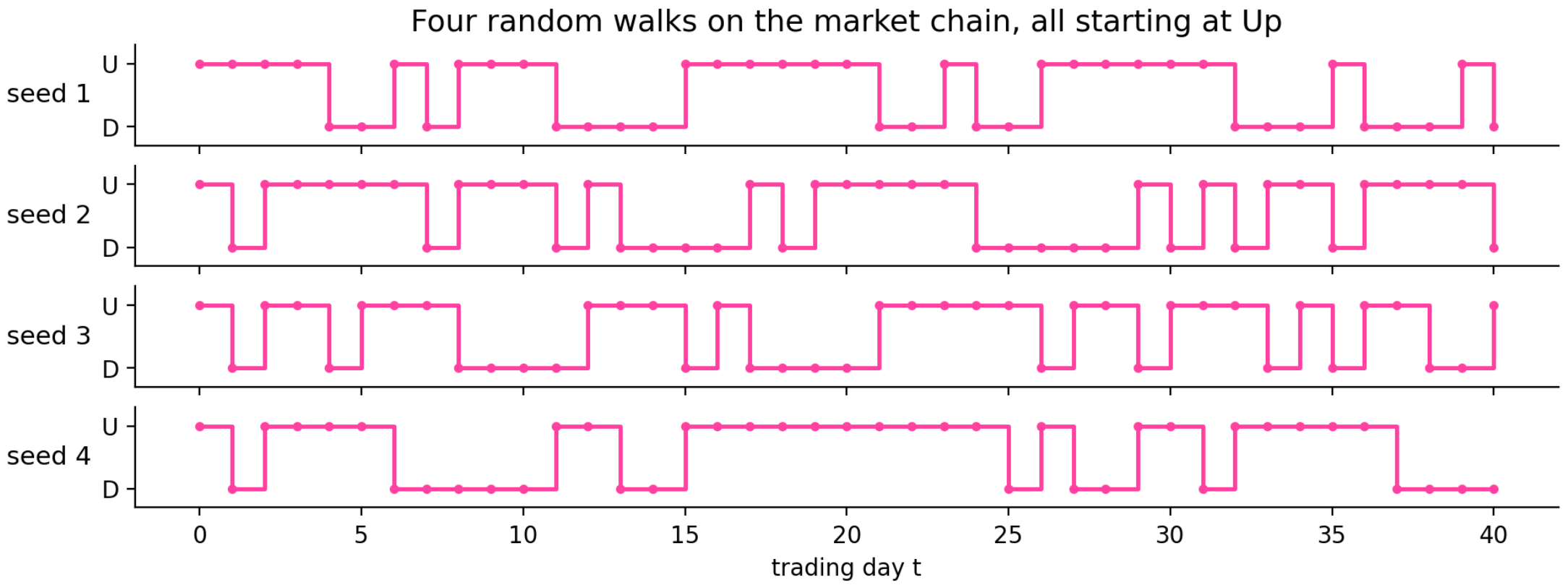
First-order chain from C

`C → F6 → F6 → Dm → A → F#m → F#m → F#m → F#m7 → F#m7 → B7 → Fdim → F#m → F#m → F#m → F#m → F#m → Gm → F → F → F`

- Both use the same 99 chords. Only the chain uses what Bach does **next**
- Listen: `bach.py` plays the original chorales, independent draws, and the order-1 and order-2 chains

Today: recap → connected chains → simulation → forecasts → stationary distribution

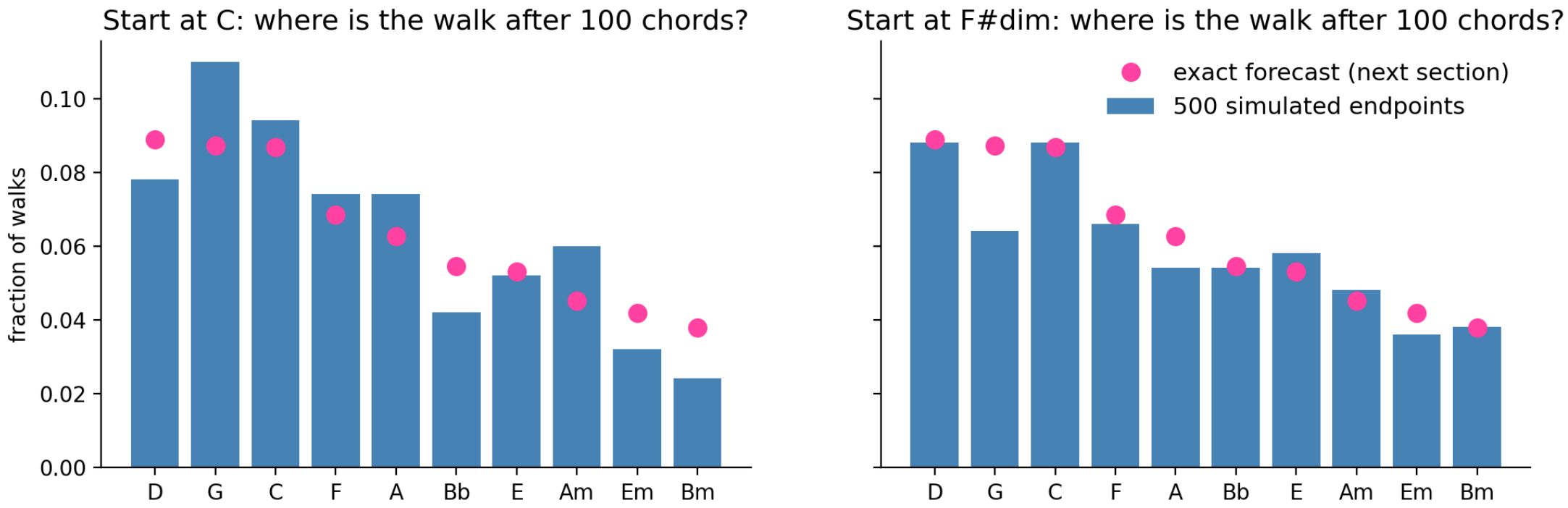
# Random Walks on the Market Chain




- Same chain, same start, different seeds: every path is different
- What stays the same across runs? Look at the long run, not at any single day

Today: recap → connected chains → simulation → forecasts → stationary distribution

# Many Walks: A Distribution Over Endpoints




- Run 500 walks of 100 chords and tabulate where each one ends: a Monte Carlo estimate of a PMF
- Starting at the common C or the rare F#dim gives nearly the same endpoint distribution. After 100 chords the start is forgotten
- Pink dots: the exact answer, which we compute next without simulating

Today: recap → connected chains → simulation → forecasts → stationary distribution

# Main idea

- To run a random walk, start in a random place, grab that row of P and randomly pick the next step, then move and do it again until you get stuck somewhere

# Forecasting and the Long Run

# Forecasting: Keep the Whole Distribution

- A simulation draws **one** path. A **forecast** keeps the probability of **every** state
- Write $\pi_t$ for the distribution at step t, as a row vector

$$\pi_{t+1} = \pi_t P, \qquad \pi_h = \pi_0 P^h$$

$$\pi_{t+1}(j) = \sum_i \pi_t(i)\, P_{ij}$$

- Add up the probability of reaching j through every possible current state i
- Start for certain in state i: $\pi_0$ is 1 at i and 0 elsewhere, and one step gives row i of P
- Probability is preserved: every $\pi_t$ sums to 1

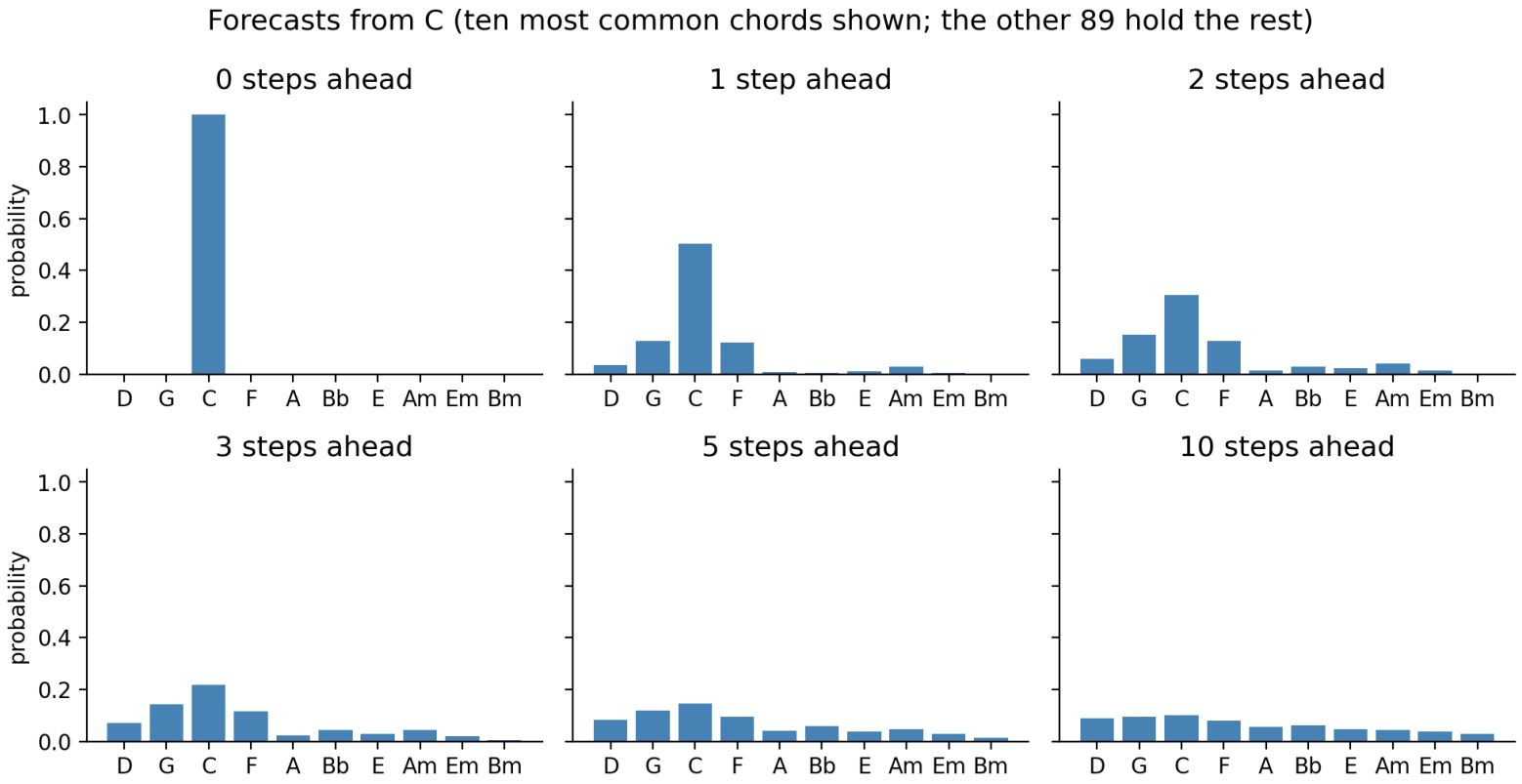
| Steps ahead h | pr[Down/flat] | pr[Up] |
|---|---|---|
| **0** | 0.0000 | 1.0000 |
| **1** | 0.4170 | 0.5830 |
| **2** | 0.4499 | 0.5501 |
| **3** | 0.4525 | 0.5475 |
| **4** | 0.4528 | 0.5472 |
| **5** | 0.4528 | 0.5472 |
| **6** | 0.4528 | 0.5472 |
| **π*** | 0.4528 | 0.5472 |

Market chain, starting at Up

Step 2 by hand: pr[Up] = 0.417 × 0.504 + 0.583 × 0.583 = 0.5501

Today: recap → connected chains → simulation → forecasts → stationary distribution

# Bach Forecasts from C




```
d = np.zeros(len(states))
d[states.index('C')] = 1

for h in range(1, 11):
    d = d @ bach_P

assert np.isclose(d.sum(), 1)
```

- At h = 1 the forecast is C's row of P
- By h = 10 the bars have stopped changing much

Today: recap → connected chains → simulation → forecasts → stationary distribution

# The Stationary Distribution

- A distribution π* is **stationary** if one more step leaves it unchanged:

$$\pi^* = \pi^* P, \qquad \sum_i \pi^*(i) = 1$$

- If the chain starts in π*, it is in π* at every later step
- Market chain: in balance, the flow Down → Up equals the flow Up → Down

$$\pi^*(D)\,(0.504) = \pi^*(U)\,(0.417)$$

$$\pi^*(U) = \frac{0.504}{0.504 + 0.417} = 0.547$$

- For a big chain, find π* numerically by **power iteration**: keep multiplying by P until nothing changes

```
pi = np.full(len(states), 1 / len(states))
for _ in range(10_000):
    new = pi @ bach_P
    if np.abs(new - pi).sum() < 1e-12:
        break
    pi = new
assert np.allclose(pi @ bach_P, pi)
```

*Also invariant, equilibrium, or steady state*

Today: recap → connected chains → simulation → forecasts → stationary distribution

# When Does the Chain Settle Down?

- **Theorem.** If a finite chain is **irreducible** and **aperiodic**, then
  - it has exactly one stationary distribution $\pi^*$, and
  - $\pi_0 P^h \to \pi^*$ as h grows, for **every** starting distribution $\pi_0$
- Equivalently, every row of P^h approaches the same row, $\pi^*$
- Not connected: different starts can settle in different places
- Absorbing states: eventually all starts end in one, but multiple absorbing states means multiple possible endings
- Periodic: forecasts can cycle forever instead of converging
- Bach order 1: connected, with self-loops, so the theorem applies

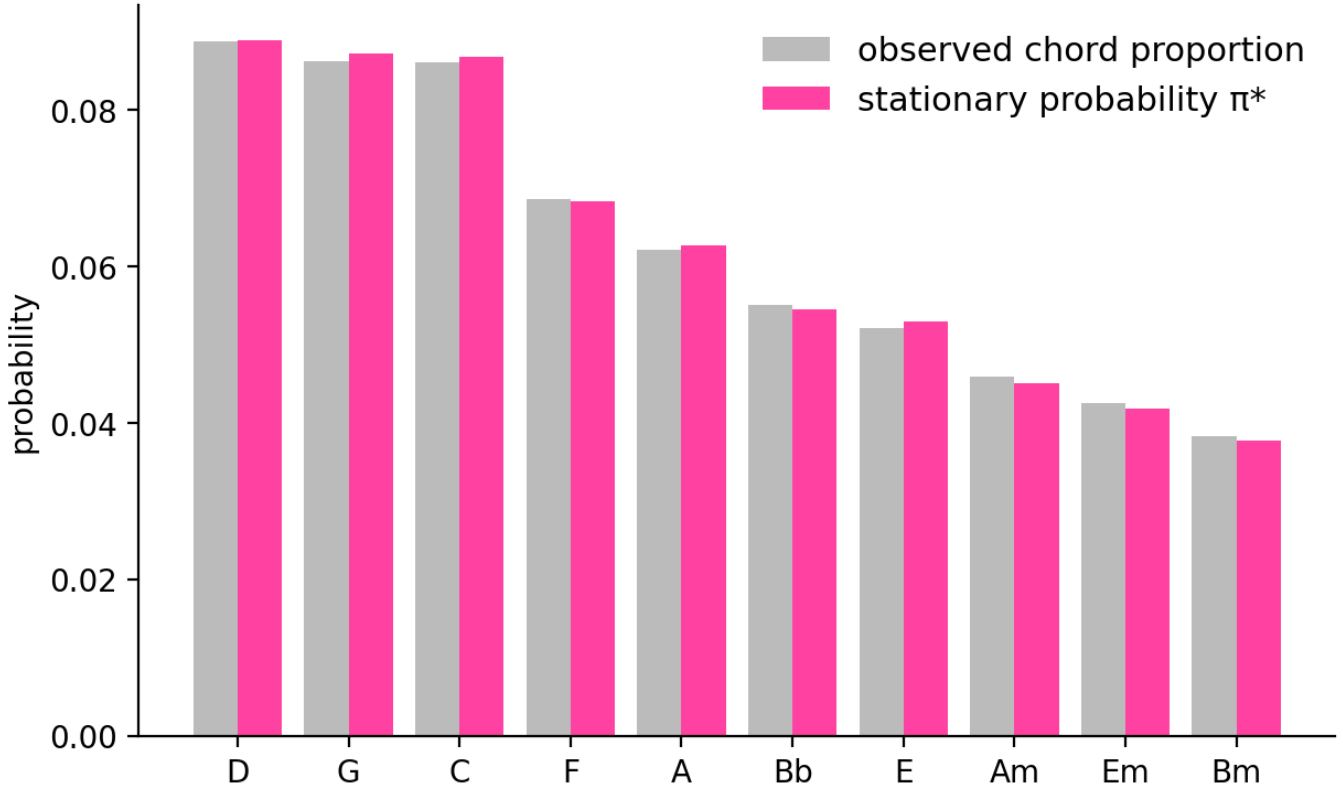
| P^h | D → D | D → U | U → D | U → U |
|---|---|---|---|---|
| **h = 1** | 0.4960 | 0.5040 | 0.4170 | 0.5830 |
| **h = 2** | 0.4562 | 0.5438 | 0.4499 | 0.5501 |
| **h = 5** | 0.4528 | 0.5472 | 0.4528 | 0.5472 |
| **h = 10** | 0.4528 | 0.5472 | 0.4528 | 0.5472 |

Both rows approach $\pi^* = (0.4528, 0.5472)$

- Row i of `np.linalg.matrix_power(P, h)` is the h-step forecast from state i

Today: recap → connected chains → simulation → forecasts → stationary distribution

# Bach: Stationary vs. Observed Proportions




- π* describes the fitted model run forever
- The observed proportions come from 62 finite chorales with beginnings and endings
- They are close but need not match exactly: chorales tend to start and end on particular chords
- Random walks are like simulated chorales, so the proportions of chords over time will also reach the stationary distribution

# Main idea

- To evolve a distribution, just keep multiplying by P until it settles down. Any starting distribution will settle into the stationary distribution if the chain is aperiodic and irreducible.

# Coding Exercise: Simulate and Forecast Taxis

- Choose a neighborhood and simulate 20 transitions with your first-order matrix. Repeat with different seeds. What changes and what stays the same?
- Simulate 500 walks of 100 transitions and plot the PMF of where they end. Repeat from another neighborhood. Similar or different?
- Forecast the location distribution after 2, 3, 5, and 10 steps with `density @ taxicab_P`. Check that it still sums to 1
- Hint: row i of `np.linalg.matrix_power(taxicab_P, h)` is the horizon-h forecast from state i
- How fast does the starting location fade? Compare a long-horizon forecast with the observed neighborhood proportions
- **Before forecasting**: is your taxi chain connected? Does any state have no departures? Decide and document a policy

# Going Further: Regular and Ergodic Chains

Optional slides for students who want more: the vocabulary behind the convergence theorem, and where it shows up in research

# Regular Chains

- A chain is **regular** if some power P^n has every entry greater than 0
- Entry (i, j) of P^n is the probability of being at j after exactly n steps from i. Regular means one fixed n works from every start to every end
- Once P^n is all positive, every later power is too: each row of P has a positive entry, and it multiplies an all-positive matrix
- Regular is stronger than connected. The flip chain is connected, but every power has zeros
- For finite chains: **regular = irreducible + aperiodic**

# Why Irreducible + Aperiodic Gives Regular

- **Step A (aperiodic).** The return times to state i are closed under addition and have gcd 1, so every n past some cutoff $N_i$ is a return time
- **Step B (irreducible).** Pick a path from i to j of length $m_{ij}$. For any $n \geq N_i + m_{ij}$: loop back to i for $n - m_{ij}$ steps, then walk to j
- **Step C (finite).** Let $n^*$ be the largest $N_i + m_{ij}$ over all pairs. Then every entry of $P^{n^*}$ is positive
- Converse: if $P^n$ and $P^{(n+1)}$ are both positive, every state returns in n and in n + 1 steps, and $\gcd(n, n + 1) = 1$
- Step C needs finitely many states: with infinitely many, the cutoffs can grow without bound

# Positive Recurrence and Ergodic Chains

- A state is **recurrent** if the chain returns to it with probability 1, and **transient** otherwise
- A recurrent state is **positive recurrent** if its mean return time m_i is finite, and **null recurrent** if m_i is infinite
- **Ergodic** = irreducible + aperiodic + positive recurrent
- A finite irreducible chain is always positive recurrent. So for finite chains: ergodic = regular = irreducible + aperiodic
- The extra condition only matters on infinite state spaces
- Terminology varies: Grinstead & Snell use 'ergodic' for irreducible alone. Check each book's definition

# The Ergodic Theorem

- For an ergodic chain:
  - There is exactly one stationary distribution $\pi^*$
  - $\pi^*_j = 1 / m_j$: states the chain returns to quickly get more weight
  - $P^h(i, j) \to \pi^*_j$ for every starting state $i$
  - The fraction of time one long walk spends in $j \to \pi^*_j$: a law of large numbers for Markov chains
- What breaks without each condition:
  - Not irreducible: several stationary distributions
  - Periodic: $\pi^*$ exists, but $P^h$ oscillates instead of converging
  - Not positive recurrent: no stationary distribution at all

# Mean First Passage Times

- The **first passage time** from i to j is the number of steps until a chain started at i first reaches j. Its expected value is the **mean first passage time** $h_{ij}$
- **First-step analysis**: condition on the first move. Then $h_{ij} = 1 + \Sigma_{k \neq j} P_{ik} h_{kj}$, with $h_{jj} = 0$
- One linear system per target state j: $(I - Q)\, h = 1$, where Q is P with row and column j removed. Solve it with np.linalg.solve
- Market chain, target Up: $h(\text{Down} \to \text{Up}) = 1 + 0.496 \times h(\text{Down} \to \text{Up})$, so $h(\text{Down} \to \text{Up}) = 1 / 0.504 = 1.984$ steps
- Target Down: $h(\text{Up} \to \text{Down}) = 1 / 0.417 = 2.398$ steps. With two states the wait is geometric: leave with probability p per step, wait 1 / p steps on average

# Mean Return Time

- The **mean return time** $m_j$ is the expected number of steps for a chain started at j to come back to j
- Take one step, then make a first passage back: $m_j = 1 + \Sigma_k P_{jk} h_{kj}$
- Market chain: $m_{Up} = 1 + 0.417 \times 1.984 = 1.827$ steps, and $m_{Down} = 1 + 0.504 \times 2.398 = 2.209$ steps
- Compare $1 / \pi^*$: $1 / 0.5472 = 1.827$ and $1 / 0.4528 = 2.209$. In general **$\pi^*_j = 1 / m_j$** (Kac's formula)
- Why: cut one long walk at each visit to j. The gaps between visits are i.i.d. with mean $m_j$, so after k visits about $k \times m_j$ steps have passed, and the fraction of time spent at j is about $1 / m_j$
- Read it both ways: a state with $\pi^* = 0.002$ is revisited on average once every 500 steps

# Detailed Balance

- A distribution $\pi$ satisfies **detailed balance** if $\pi_i P_{ij} = \pi_j P_{ji}$ for every pair of states: the probability flow from i to j equals the flow from j back to i
- Sum both sides over i: $\Sigma_i \pi_i P_{ij} = \pi_j \Sigma_i P_{ji} = \pi_j$. So detailed balance implies that $\pi$ is stationary
- The converse fails. Stationarity (**global balance**) only needs the total flow into each state to equal the total flow out
- Market chain: $0.4528 \times 0.504 = 0.2282 = 0.5472 \times 0.417$. Every irreducible two-state chain satisfies detailed balance
- Counterexample: a 3-cycle A → B → C → A, stepping forward with probability 0.8 and back with 0.2. $\pi^*$ is uniform, but the flow A → B is 0.8 / 3 and the flow B → A is only 0.2 / 3
- A chain with detailed balance is **reversible**: run backward in time, it has the same statistics. MCMC uses detailed balance to make a target $\pi$ stationary

# Weak vs. Strong Law of Large Numbers

- Earlier in the course, the **weak law (WLLN)**: for any $\varepsilon > 0$, $\Pr(|\bar{X}_n - \mu| > \varepsilon) \to 0$ as n grows
- The **strong law (SLLN)**: $\Pr(\bar{X}_n \to \mu \text{ as } n \to \infty) = 1$
- The WLLN is a statement about each n separately: at a large n, few runs are far from μ
- The SLLN is a statement about each whole run: with probability 1, the sequence of running averages converges and stays converged
- The WLLN alone would allow every run to wander away from μ now and then, forever, as long as few runs are away at any given n. The SLLN rules that out
- Strong implies weak. For i.i.d. samples with a finite mean, both hold

# The Ergodic Theorem Is a Strong Law for Markov Chains

- SLLN: if $X_1, X_2, \ldots$ are i.i.d., then $(1/n)\ \Sigma_t\ f(X_t) \to E[f(X)]$ with probability 1
- Ergodic theorem: if $X_t$ is an irreducible, positive recurrent chain, then from any start $(1/n)\ \Sigma_t\ f(X_t) \to \Sigma_j\ \pi^*_j\ f(j)$ with probability 1
- What changed: the samples are dependent, and the limit is the average under $\pi^*$, not under the starting distribution
- Taking f = 1 at state j and 0 elsewhere gives the familiar version: the fraction of time spent at $j \to \pi^*_j$
- i.i.d. sampling is a special case: if every row of P is the same distribution p, the $X_t$ are i.i.d. draws from p, $\pi^* = p$, and the ergodic theorem is the SLLN
- Aperiodicity is not needed for time averages: a periodic chain's $P^h$ oscillates, but its time averages still converge

# Proof Sketch: Independence One Level Up

- Fix a state i and cut the walk at every visit to i. Each piece starts fresh at i, so the pieces (**cycles**) are i.i.d.: the **strong Markov property**
- For cycle k, let $\tau_k$ be its length and $Y_k$ the sum of $f(X_t)$ over the cycle. The pairs $(\tau_k, Y_k)$ are i.i.d.
- Apply the ordinary SLLN twice: $(1/K)\,\Sigma_k Y_k \to E[Y]$ and $(1/K)\,\Sigma_k \tau_k \to E[\tau] = m_i$
- So the time average after K cycles, $\Sigma_k Y_k / \Sigma_k \tau_k$, converges to $E[Y] / m_i$
- A cycle visits j on average $\pi^*_j / \pi^*_i = \pi^*_j m_i$ times, so $E[Y] = m_i \Sigma_j \pi^*_j f(j)$, and the limit is $\Sigma_j \pi^*_j f(j)$
- Single steps are dependent, but whole cycles are independent, so the SLLN applies to cycles. The steps before the first visit and the unfinished last cycle do not change the limit

# How Accurate Is a Time Average?

- The ergodic theorem says time averages converge. The **Markov chain CLT** says how fast: the error shrinks like $1 / \sqrt{n}$, as for i.i.d. samples
- Dependence changes the constant: $\text{Var}(\text{time average}) \approx \sigma^2 (1 + 2 \Sigma_k \rho_k) / n$, where $\rho_k$ is the correlation between $f(X_t)$ and $f(X_{t+k})$
- So n correlated steps carry about as much information as $n / (1 + 2 \Sigma_k \rho_k)$ independent draws: the **effective sample size**
- Market chain, f = 1 on Up: $\rho_k = 0.079^k$, where $0.079 = 0.583 - 0.504$. So $1 + 2 \Sigma_k \rho_k = 1.079 / 0.921 = 1.17$
- Check: over 4,000 simulated runs of 1,000 steps, the SD of the fraction of Up steps was 0.0169. The formula gives 0.0170; treating the steps as i.i.d. gives 0.0157
- Sticky chains ($\rho_k$ near 1) have much smaller effective sample sizes, which is why MCMC users check autocorrelation

# Extension: Continuous-Time Markov Chains

- In continuous time the chain can jump at any moment. The Markov property still holds: the future depends only on the current state, not on how long the chain has been there
- That forces each holding time to be **exponential**, the only memoryless continuous distribution: the chain stays in state i for an Exp($q_i$) time, with mean $1 / q_i$
- When it leaves i, it jumps to j with probability $P_{ij}$. These jumps form a discrete-time chain, the **jump chain**
- The **rate matrix** Q combines both: $q_{ij} = q_i P_{ij}$ is the rate of jumps $i \to j$, and $q_{ii} = -q_i$, so every row sums to 0
- Transition probabilities over a time t: $P(t) = e^{(tQ)}$, the matrix exponential (scipy.linalg.expm). It plays the role of $P^h$. Over a short step $\Delta t$, $P(\Delta t) \approx I + Q\,\Delta t$
- Examples: queues with Poisson arrivals, chemical reactions (Gillespie simulation), and DNA substitution models in phylogenetics (Jukes–Cantor: a 4-state chain on A, C, G, T)

# The Long Run in Continuous Time

- Stationary distribution: $\pi Q = 0$, so the net probability flow into every state is zero. This replaces $\pi P = \pi$
- A finite irreducible chain in continuous time has a unique $\pi$, and $P(t) \to \pi$ from every start as $t \to \infty$. Periodicity cannot occur, because jumps happen at random times
- Detailed balance becomes $\pi_i q_{ij} = \pi_j q_{ji}$, and the ergodic theorem still holds: the fraction of time spent in $j \to \pi_j$
- Time weights differ from jump-chain weights: $\pi_j$ is proportional to $\psi_j / q_j$, where $\psi$ is the jump chain's stationary distribution. States held longer get more time
- Two-state example: Off → On at rate 2 per hour, On → Off at rate 1 per hour. $\pi = (1/3, 2/3)$, and $\pi Q = 0$ checks out
- Starting Off, pr[Off] = 0.827 at t = 0.1 h, 0.482 at 0.5 h, 0.367 at 1 h, and 0.335 at 2 h. The gap to 1/3 shrinks like e^(–3t), halving every 0.23 hours

# Hidden Markov Models

- In a **hidden Markov model (HMM)** the chain $X_t$ is not observed. You see $Y_t$, which depends only on the current state through the **emission probabilities** $B_{ik} = \Pr[Y_t = k \mid X_t = i]$
- Example, the dishonest casino (Durbin et al., 1998): the dealer switches between a fair die and a loaded die that rolls 6 half the time. Fair → loaded with probability 0.05, loaded → fair with 0.10. You only see the rolls
- The hidden chain is ergodic with $\pi^* = (2/3 \text{ fair}, 1/3 \text{ loaded})$, so in the long run the fraction of 6s is $2/3 \times 1/6 + 1/3 \times 1/2 = 0.278$
- **Filtering**: carry the distribution of the hidden state forward, as in our forecasts, but reweight by each observation: $\alpha_{t+1} = (\alpha_t P) \times B[:, y_{t+1}]$, then normalize
- Starting from $\pi^*$: one 6 gives $\Pr[\text{loaded}] = 0.60$; five 6s in a row give $\Pr[\text{loaded}] = 0.94$
- The same recursion (the **forward algorithm**) gives the likelihood of the data in $n \times S^2$ steps instead of $S^n$. **Viterbi** finds the most likely hidden path, and **Baum–Welch** (an EM algorithm) fits P and B when the states are never seen
- Uses: speech recognition (Rabiner, 1989), gene finding, regime-switching models in economics (Hamilton, 1989), and inferring cell states from noisy markers

# Coupling: The Kruskal Count Card Trick

- Shuffle **one** deck and deal it face up, one card at a time, while both players watch the same cards. Count A = 1, 2–10 at face value, J/Q/K = 10
- Each player secretly picks a start among the first 10 cards. Look at its value v, jump v cards ahead, and repeat until the jump would pass the end
- Once two players land on the same card they read the same values from then on, so their paths **merge** and they end on the same card
- Each jump by the trailing player has about a 1-in-6.5 chance of landing on a card the leader used, so the chance of never meeting shrinks geometrically
- This is **coupling**, the standard proof of the convergence theorem: run two copies from different starts, show they meet with probability 1
- The trick: you name your friend's last card without knowing her start. It only works because both of you read the same cards. With separate decks the paths cannot merge

```
Simulated, uniform shuffles
(faces = 10)

decks  pr[two players match]
  1          0.71
  2          0.92
  4          0.99

One deck, all 10 starts
end on the same card:  0.29
Separate decks match:   0.12

Periodic version
Every card worth 2:
odd and even starts never meet
```

# In Research: Google's PageRank

- Brin & Page model a **random surfer**: at each step, follow a random link on the current page
- The raw link chain is not irreducible, and pages with no outgoing links are dead ends: the same zero-row problem as the Bach chain's final chord pairs
- Fix: with probability 0.85 follow a link, and with probability 0.15 jump to a uniformly random page (this makes the chain regular by design)
- Every entry of the new matrix is at least $0.15 / N > 0$, so P itself is positive: **regular by construction**
- Regularity guarantees a unique $\pi^*$, and power iteration converges to it. A page's rank is its $\pi^*$

# In Research: Cancer Cell States

- Gupta et al. sorted breast-cancer cells (lines SUM149 and SUM159) into three states: stem-like, basal, and luminal
- Each pure subpopulation, grown on its own, drifted back to the same mix of the three states over several days
- They fitted a **3-state Markov chain**: each cell switches state with fixed probabilities per time step
- The chain is **ergodic**, so the population converges to $\pi^*$ from any starting mix: the 'phenotypic equilibrium' of the title
- Implication for treatment: killing one state is not enough if the others regenerate it. Thursday's tumor-heterogeneity lab asks the same kind of question

## Source

Gupta, P. B., Fillmore, C. M., Jiang, G., et al. (2011). Stochastic state transitions give rise to phenotypic equilibrium in populations of cancer cells. **Cell**, 146(4), 633–644.

# In Research: Markov Chain Monte Carlo

- Goal: draw samples from a distribution π that is too complicated to sample directly, such as a Bayesian posterior or, in 1953 when this was developed, the positions of interacting molecules
- Metropolis et al. build a chain whose stationary distribution is π by design: propose a move, then accept it with a probability chosen so the flows i → j and j → i balance under π
- Rejected proposals keep the chain in place. These self-loops make it **aperiodic**
- The ergodic theorem then says averages over one long walk converge to averages under π
- This reverses today's question: instead of finding π* for a given P, choose P so that π* is the target

**the Metropolis algorithm**

Metropolis, N., Rosenbluth, A. W., Rosenbluth, M. N., Teller, A. H. & Teller, E. (1953). Equation of state calculations by fast computing machines. **J. Chemical Physics**, 21(6), 1087–1092.

Hastings, W. K. (1970). Monte Carlo sampling methods using Markov chains and their applications. **Biometrika**, 57(1), 97–109.

# Further Reading

- **Grinstead & Snell**, Introduction to Probability, ch. 11 (free online): regular and absorbing chains, with many worked examples
- **Levin, Peres & Wilmer**, Markov Chains and Mixing Times, 2nd ed. (AMS, 2017; free PDF from the authors): how fast chains forget their start, including card shuffling and coupling
- **Norris**, Markov Chains (Cambridge, 1997): proofs of the ergodic theorem, recurrence, and transience
- **Diaconis** (2009). The Markov chain Monte Carlo revolution. Bulletin of the AMS, 46(2), 179–205: a readable survey that ties these ideas together
- **Lagarias, Rains & Vanderbei** (2009). The Kruskal count. In The Mathematics of Preference, Choice and Order (Springer), 371–391: the card trick, analyzed